

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 6

#### End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

|                     |                                               |                |
|---------------------|-----------------------------------------------|----------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester<br>V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY:<br>2026–27 |

## 1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

## 2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the  $(N, T, F)$  input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

## 3. Dataset and Experimental Setup

### Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING\_UPSTAIRS
- WALKING\_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

**Recommended laboratory subset:** To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

**Important:** The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

## Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

| Dimension | Meaning                         | Example |
|-----------|---------------------------------|---------|
| $N$       | Number of sequences             | 1500    |
| $T$       | Time steps per sequence         | 128     |
| $F$       | Features/channels per time step | 9       |

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

## 4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

## 5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as  $128 \times 9$ .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

**Suggested split:**

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

## Expected Preprocessing Output

Students must print:

Input tensor shape:

```
Training : (N_train, 128, 9)
Validation : (N_val, 128, 9)
Testing : (N_test, 128, 9)
```

Number of classes: 6  
Number of features per time step: 9  
Sequence length: 128

The actual values of  $N_{\text{train}}$ ,  $N_{\text{val}}$  and  $N_{\text{test}}$  depend on the selected subset.

## 6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

**Plot 1: Sensor signal versus time**

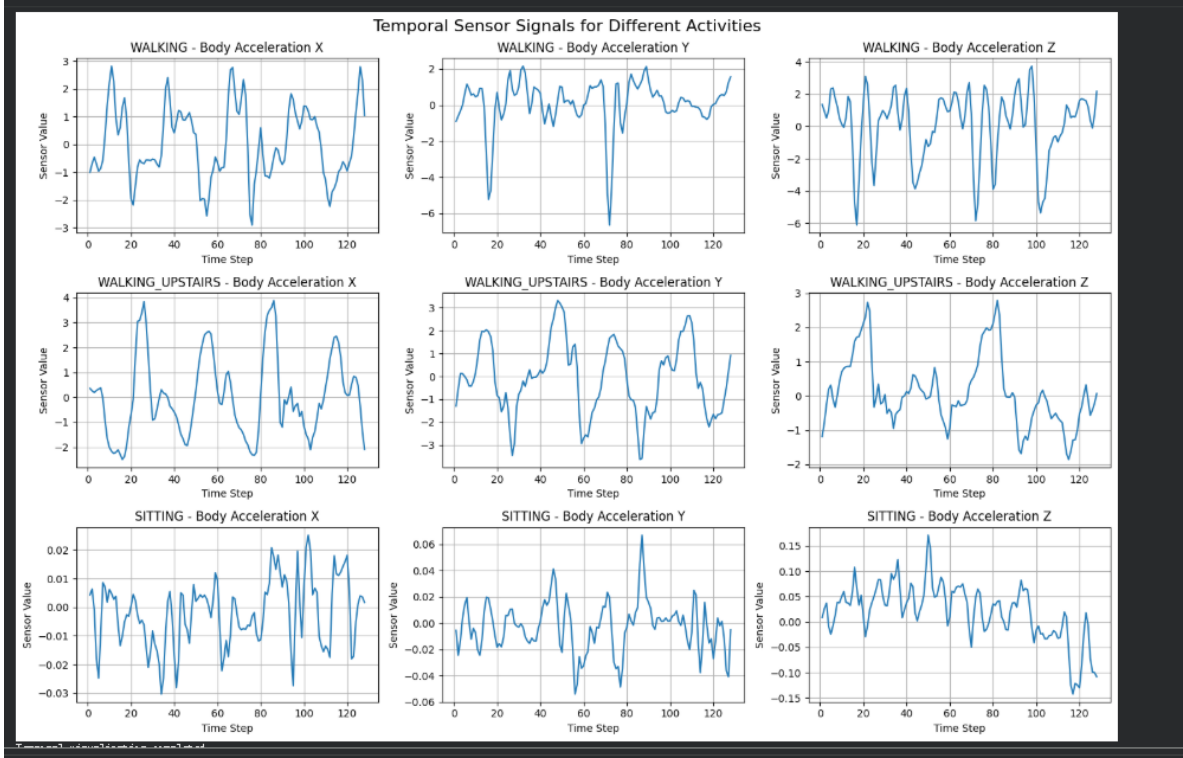


Figure 1: Representative sensor signals for different activity classes.

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

1. **Temporal pattern:** The signals show repeated peaks and valleys over time.
2. **Similar activities:** Walking and walking upstairs show similar patterns and sitting shows smaller variations.
3. **Importance of ordering:** The order of measurements is important because it shows how the signal changes with time. shuffling the data would remove the temporal pattern.

## 7. Vanilla RNN Architecture

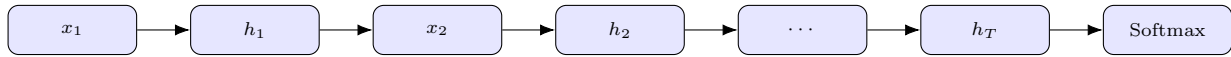
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

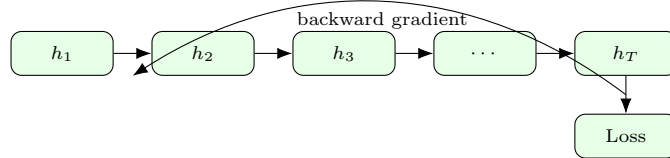
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

## 8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

### Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

### Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

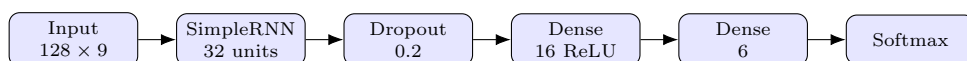
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate  $h_1$ ,  $h_2$  and  $h_3$  manually.

Students must compare their calculated values with the values obtained from a program.

## 9. RNN Implementation

Use a small recurrent architecture:



**Suggested training configuration:**

| Parameter       | Value                            |
|-----------------|----------------------------------|
| Recurrent units | 32                               |
| Dropout         | 0.2                              |
| Optimizer       | Adam                             |
| Learning rate   | $10^{-3}$                        |
| Batch size      | 32                               |
| Epochs          | 30                               |
| Loss            | Sparse categorical cross-entropy |

## 10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

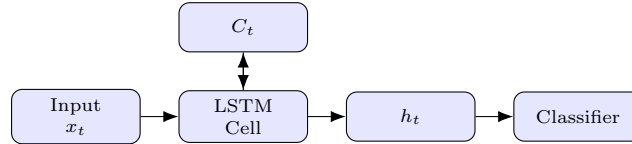
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



## 11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

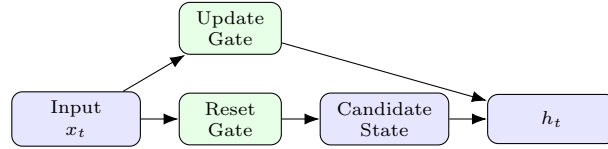
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

#### Inference:

RNN uses only one hidden state, so it can have difficulty in remembering long term information. LSTM uses one cell state and three gates. GRU uses two gates and has simpler structure than LSTM, but can capture long-term dependencies.

## 12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

| Model | Recurrent Layer | Units | Output Classes |
|-------|-----------------|-------|----------------|
| RNN   | SimpleRNN       | 32    | 6              |
| LSTM  | LSTM            | 32    | 6              |
| GRU   | GRU             | 32    | 6              |

All other experimental conditions should remain unchanged.

## 13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

#### Plot 2: Training and Validation Loss

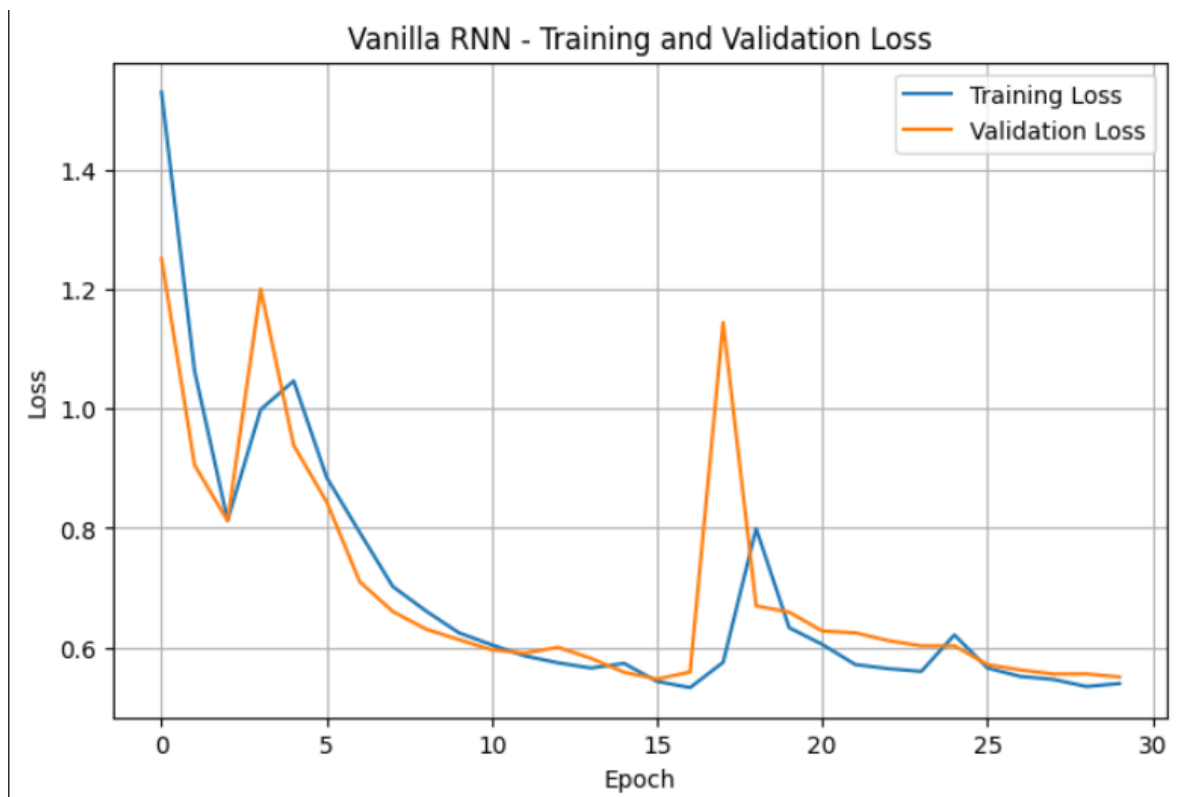


Figure 2: RNN training and validation loss.

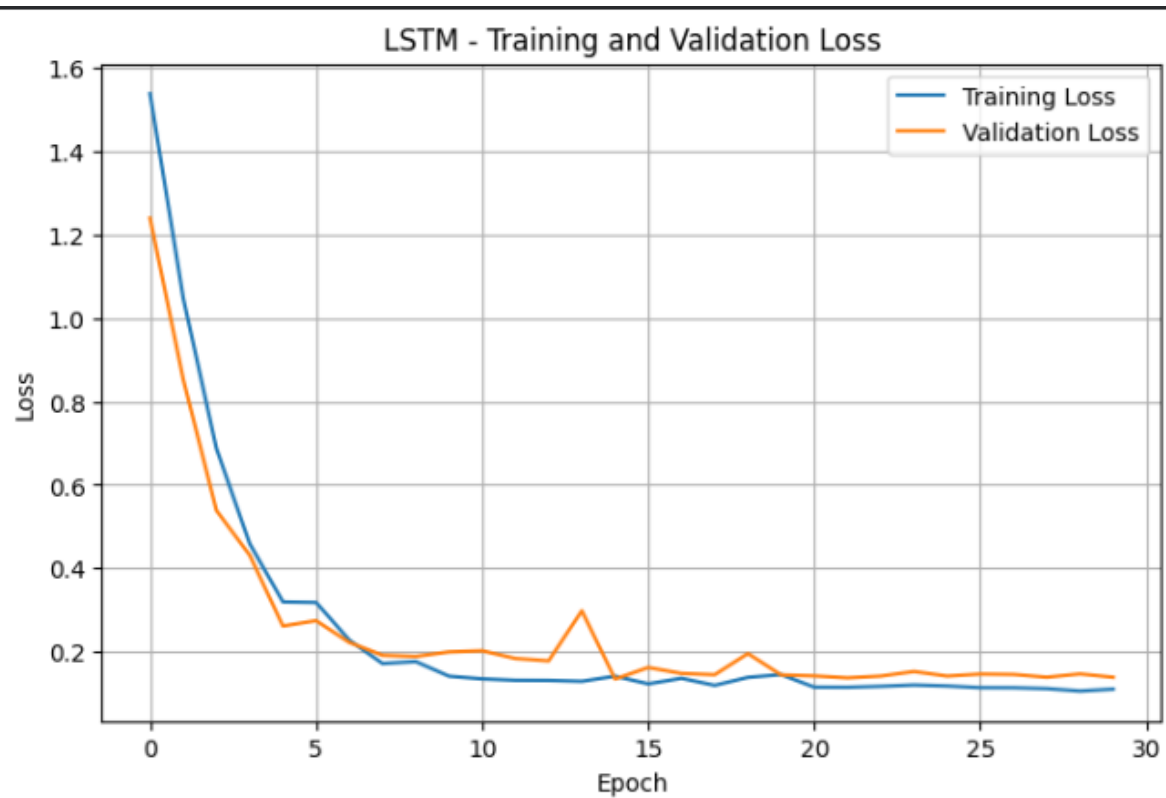


Figure 3: LSTM training and validation loss.

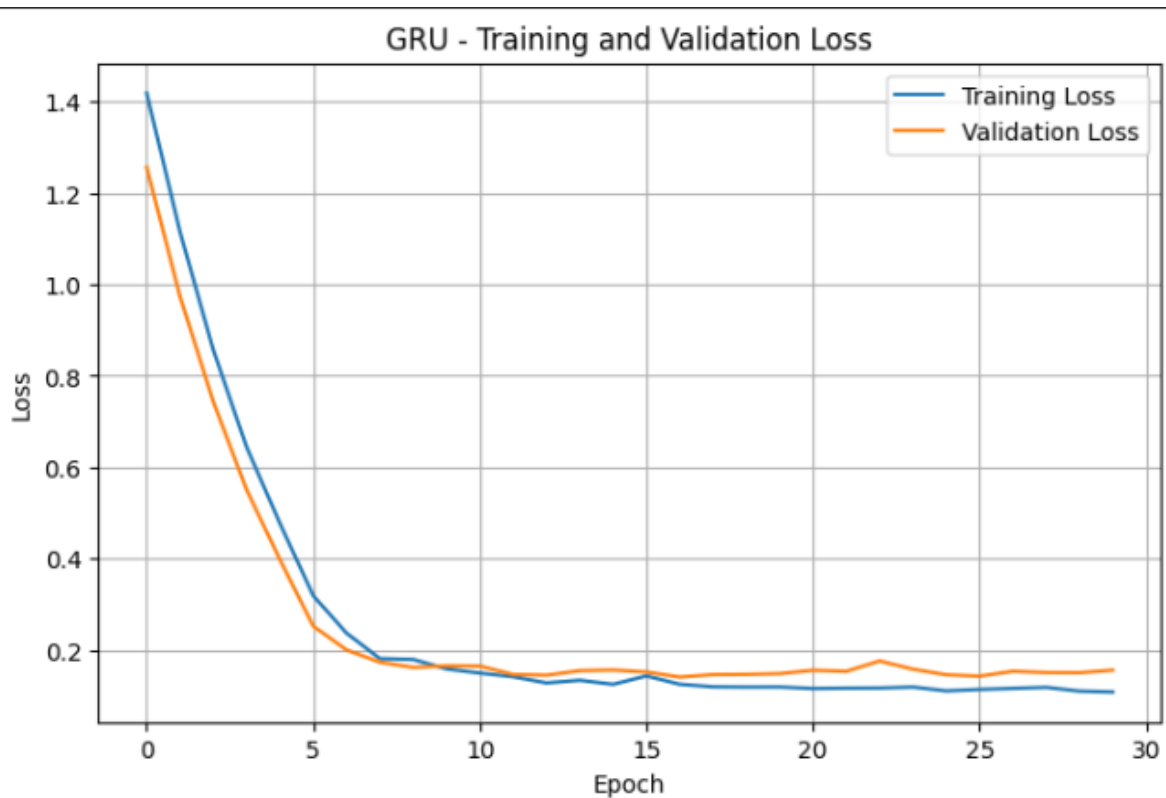


Figure 4: GRU training and validation loss.

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

**Inference:**

training loss decreases with epochs, showing that the RNN is learning. The validation loss also decreases and becomes more stable towards the end. training loss decreases as the epochs increase. The validation loss also decreases and stays close to the training loss, showing good learning. training loss decreases with the epochs and becomes stable towards the end. validation loss also decreases, showing that the GRU is learning properly.

Plot 3: Training and Validation Accuracy

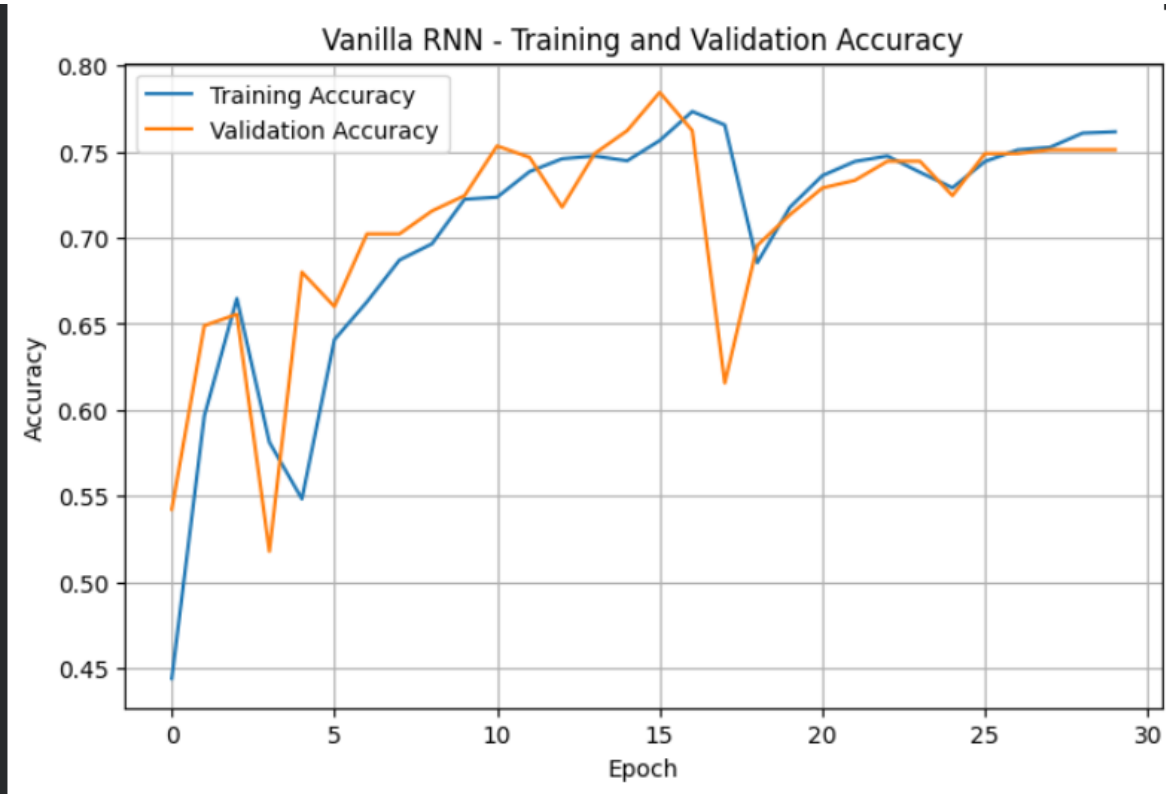


Figure 5: RNN training and validation accuracy.

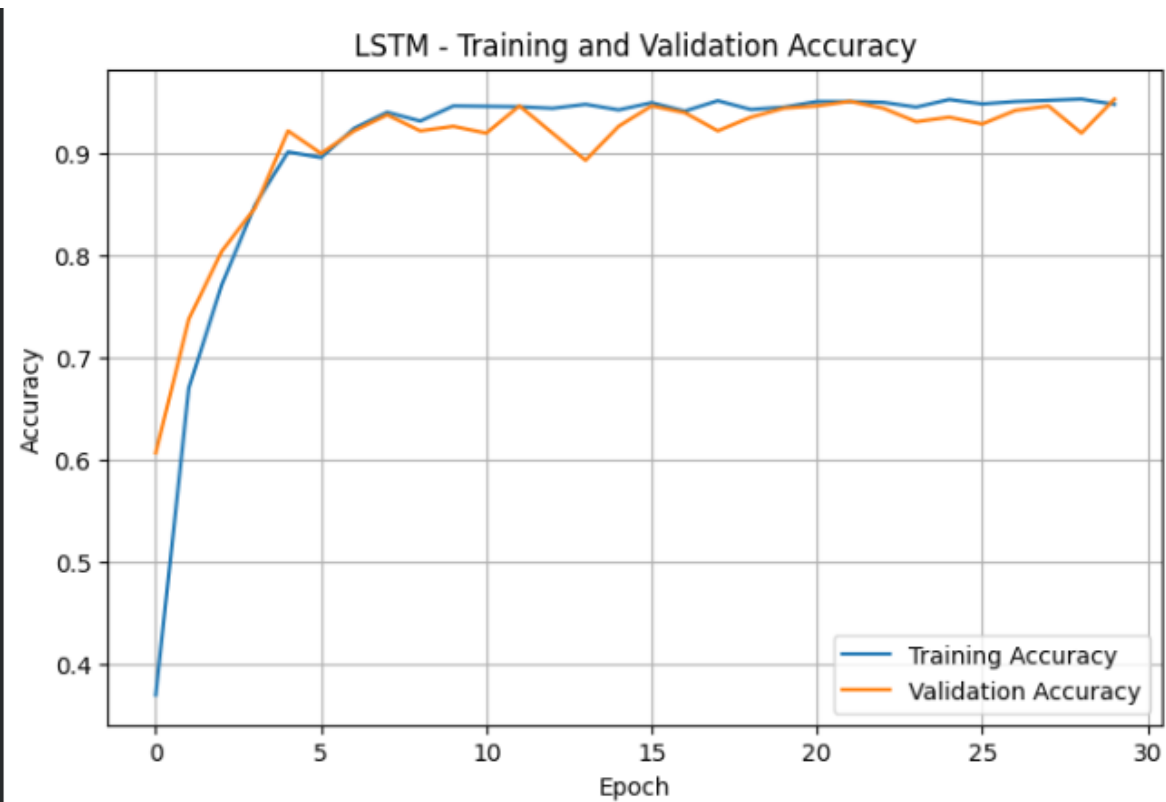


Figure 6: LSTM training and validation accuracy.

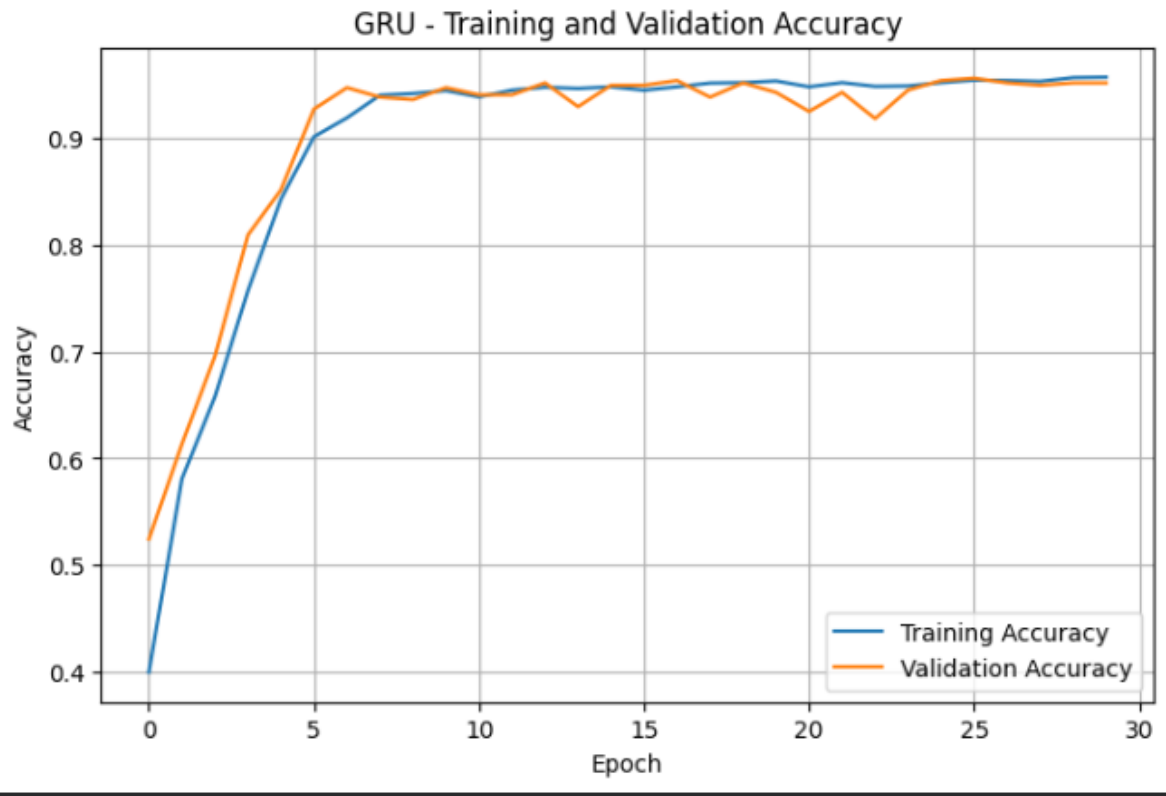


Figure 7: GRU training and validation accuracy.

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

#### Inference:

training accuracy increases with epochs and becomes more stable later. The validation accuracy also improves, showing that the RNN is learning the activity patterns. training accuracy increases with epochs and reaches a high value. The validation accuracy also increases, showing that LSTM performs well on the data. training accuracy increases with the epochs and becomes stable later. The validation accuracy also improves and shows good performance.

## 14. Performance Evaluation

Evaluate each model on the independent test set.

| Metric              | RNN     | LSTM    | GRU     |
|---------------------|---------|---------|---------|
| Accuracy (%)        | 71.3946 | 89.5826 | 89.7184 |
| Macro Precision (%) | 71.1251 | 89.5446 | 89.7931 |
| Macro Recall (%)    | 70.2833 | 89.7946 | 89.8419 |
| Macro F1 (%)        | 70.5446 | 89.6194 | 89.7424 |
| Parameters          | 1974    | 6006    | 4758    |
| Training Time (s)   | 33.9322 | 35.3816 | 30.6476 |

#### Inference:

RNN achieves 71.3946LSTM achieves 89.5826GRU achieves 89.7184

## 15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

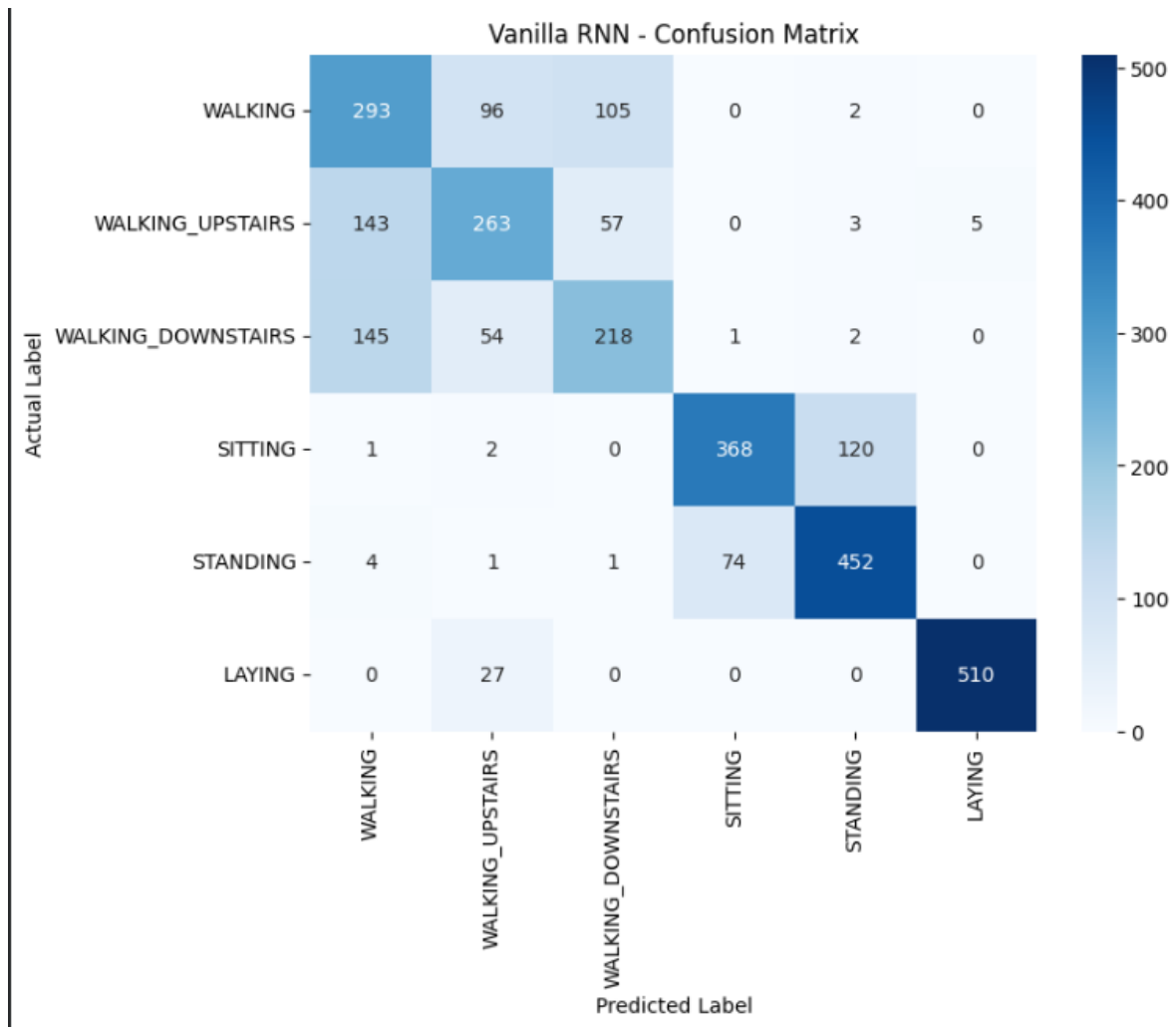


Figure 8: RNN confusion matrix.

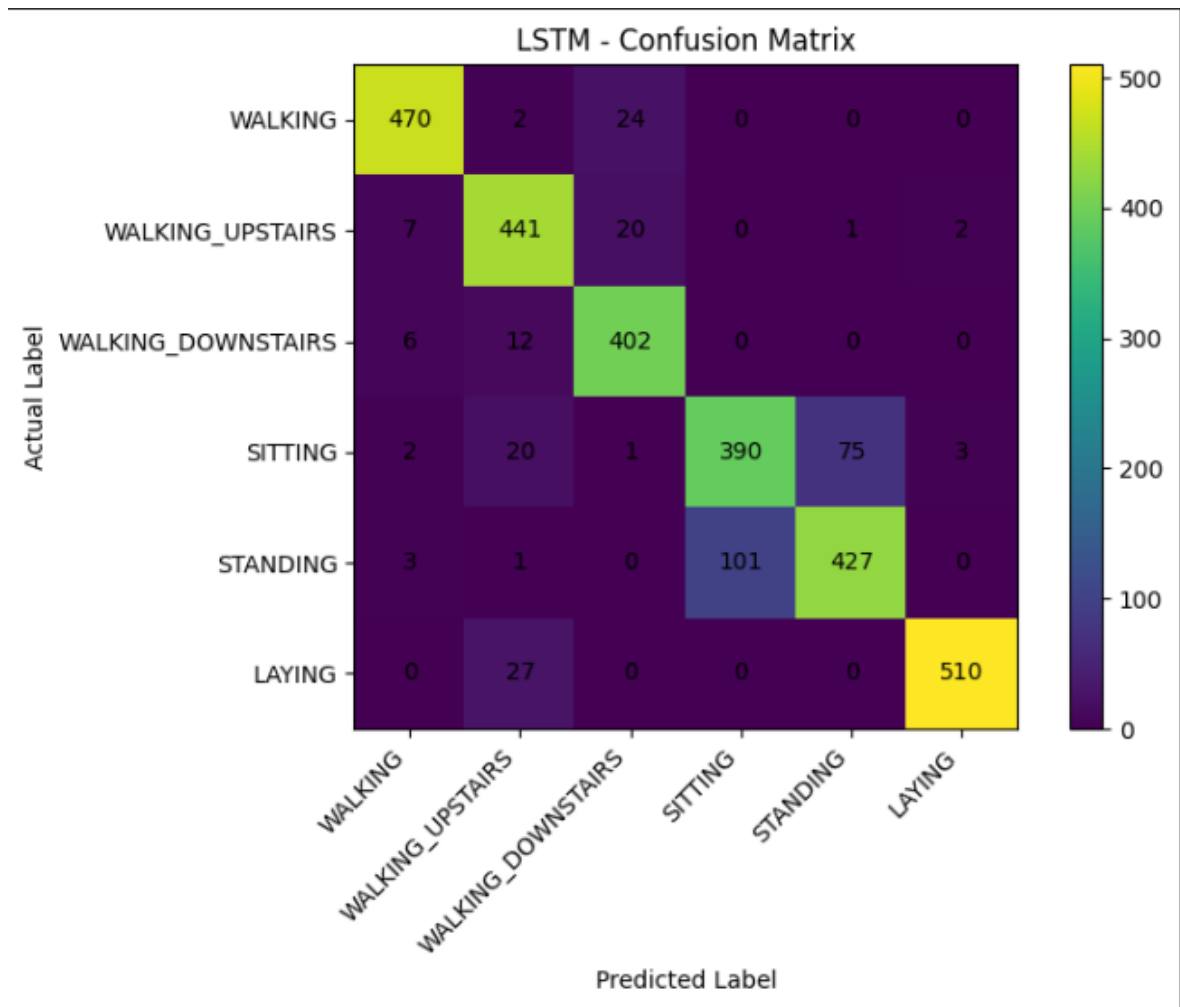


Figure 9: LSTM confusion matrix.

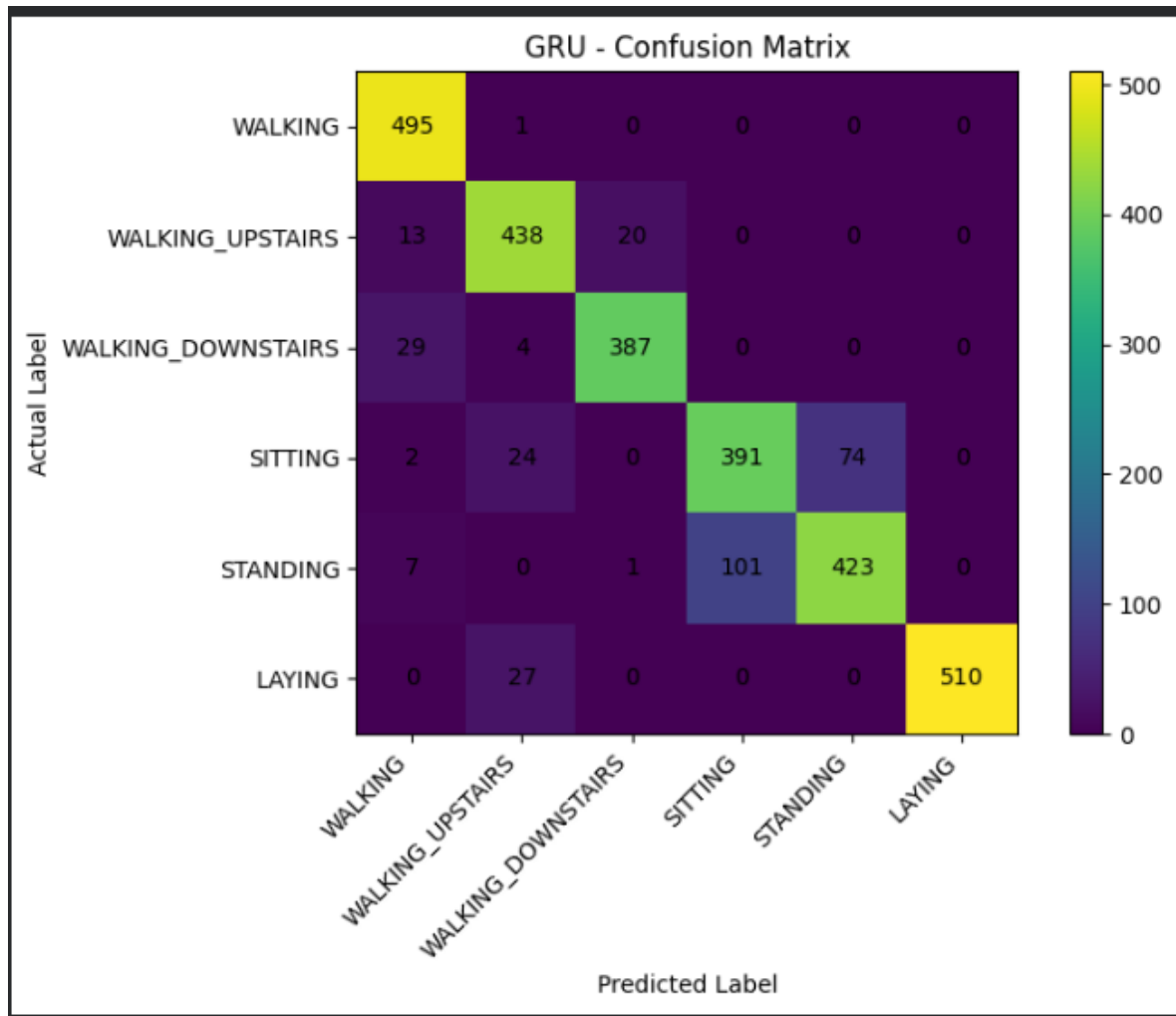


Figure 10: GRU confusion matrix.

**Inference:**

**Inference:**

diagonal values show the activities correctly predicted by the RNN. some activities are confused with other classes, showing misclassification. most of the values are concentrated on the diagonal, showing correct predictions. some small values outside the diagonal show confusion between a few activities. diagonal values show that most activities are classified correctly. There are some misclassifications between activities with similar patterns.

## 16. RNN vs. LSTM vs. GRU Comparison

| Property          | RNN     | LSTM    | GRU     |
|-------------------|---------|---------|---------|
| Hidden state      | Yes     | Yes     | Yes     |
| Cell state        | No      | Yes     | No      |
| Forget gate       | No      | Yes     | No      |
| Input gate        | No      | Yes     | No      |
| Output gate       | No      | Yes     | No      |
| Update gate       | No      | No      | Yes     |
| Reset gate        | No      | No      | Yes     |
| Parameters        | 1974    | 6006    | 4758    |
| Training time (s) | 33.9322 | 35.3816 | 30.6476 |
| Test F1-score (%) | 70.5446 | 89.6194 | 89.7424 |

Plot 5: Model Performance Comparison

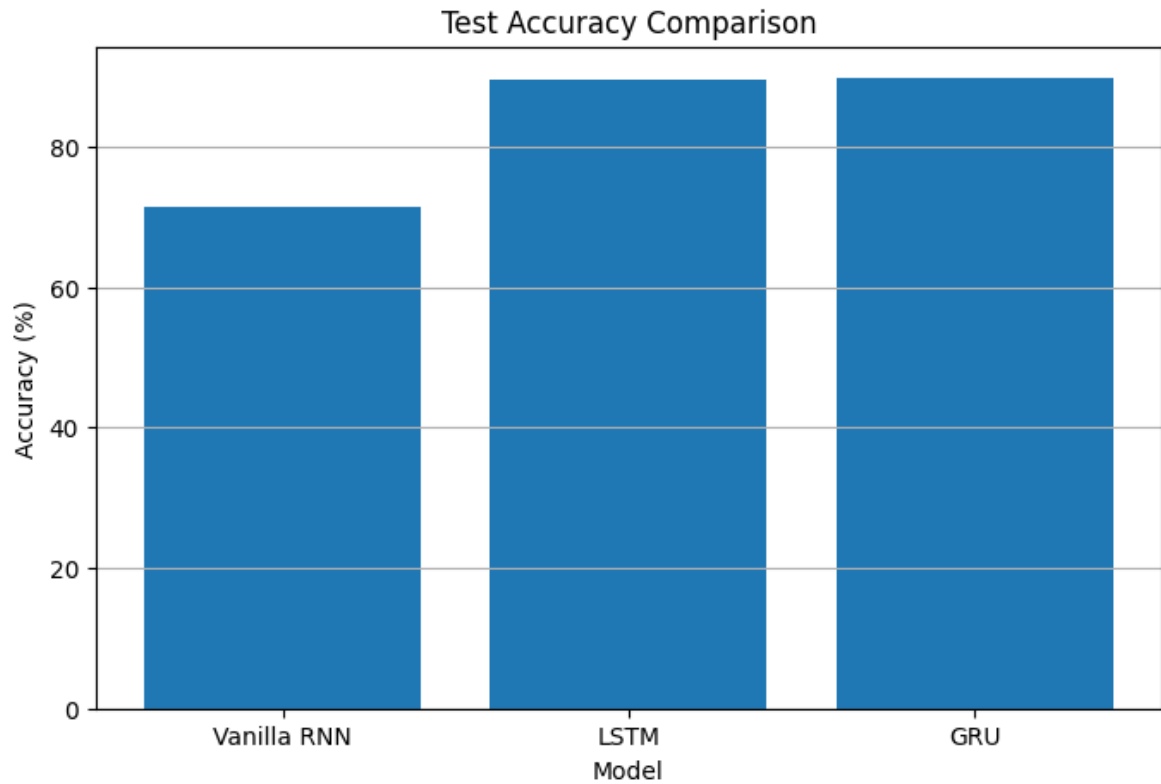


Figure 11: Comparison of RNN, LSTM and GRU performance.

#### Inference:

RNN gives lower performance compared to LSTM and GRU. LSTM and GRU give almost similar accuracy but GRU has fewer parameters and takes less training time so GRU is preferred.

## 17. Effect of Sequence Length

| Sequence Length | RNN F1  | LSTM F1 | GRU F1  |
|-----------------|---------|---------|---------|
| 32              | [value] | [value] | [value] |
| 64              | [value] | [value] | [value] |
| 128             | [value] | [value] | [value] |

Plot 6: Sequence Length vs. Test F1-score



Figure 12: Effect of sequence length on test F1-score.

**Inference:**

The F1-score changes with different sequence lengths. The sequence length with higher F1-score gives better performance for this experiment.

## 18. Video Understanding Using CNN and RNN

**Inference:**

CNN extracts the features from each video frame and the recurrent model learns the sequence of frames. This helps the model understand the action happening in the video.

Therefore, the overall pipeline can be interpreted as:

Video → Frames → Spatial Features → Temporal Features → Action Classification.

This approach allows the CNN to focus on “what is present in a frame” and the recurrent network to learn “how the visual information changes over time.”

## 19. Expected Video Input and Output

**Inference:**

10 frames are taken from each video and converted into feature vectors using CNN. These features are given to the recurrent model to predict the action class.

## 20. CNN Feature Extraction

**Required observation:**

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

**Inference:**

CNN extracts useful features from each frame of the video. These features are then given as a sequence to the RNN, LSTM or GRU model.

## 21. CNN-LSTM / CNN-GRU Model

Plot 7: Video Sample Frames

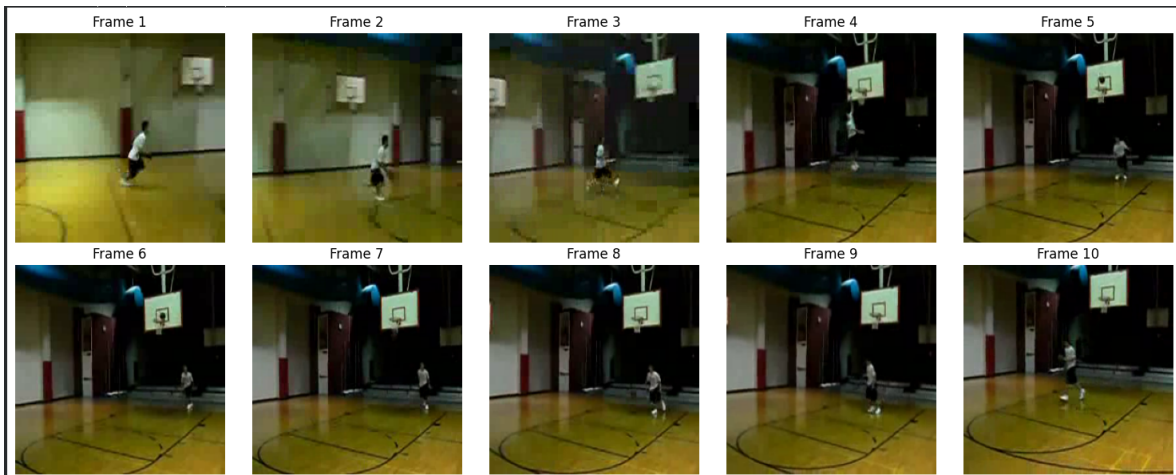


Figure 13: Representative sampled frames from a video sequence.

### Inference:

The sampled frames show the action at different time steps in the video. These frames are used by CNN to extract the visual features.

Plot 8: Video Training and Validation Curves

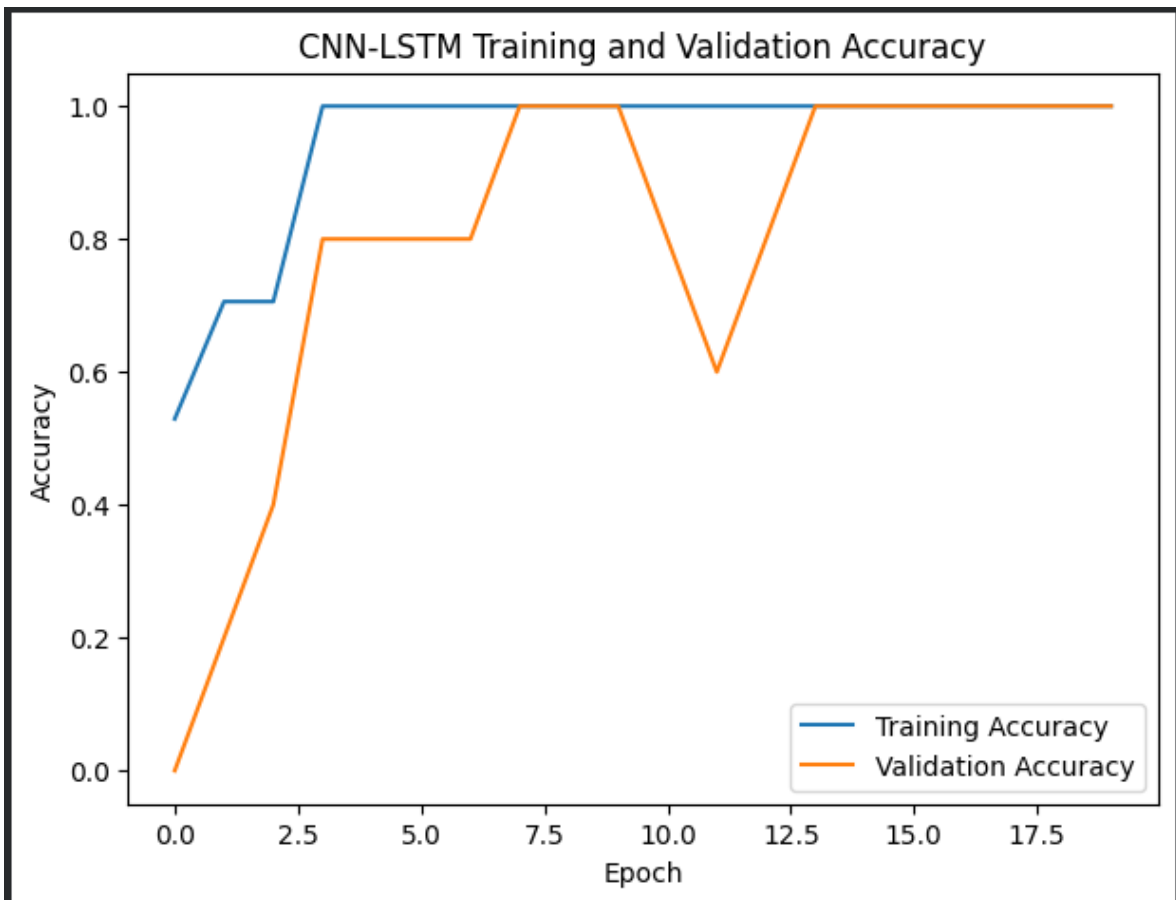


Figure 14: Video model training and validation curves.

### Inference:

training and validation curves show how the video model learns with epochs. We can see that the curves became more stable with training.

Plot 9: Video Confusion Matrix

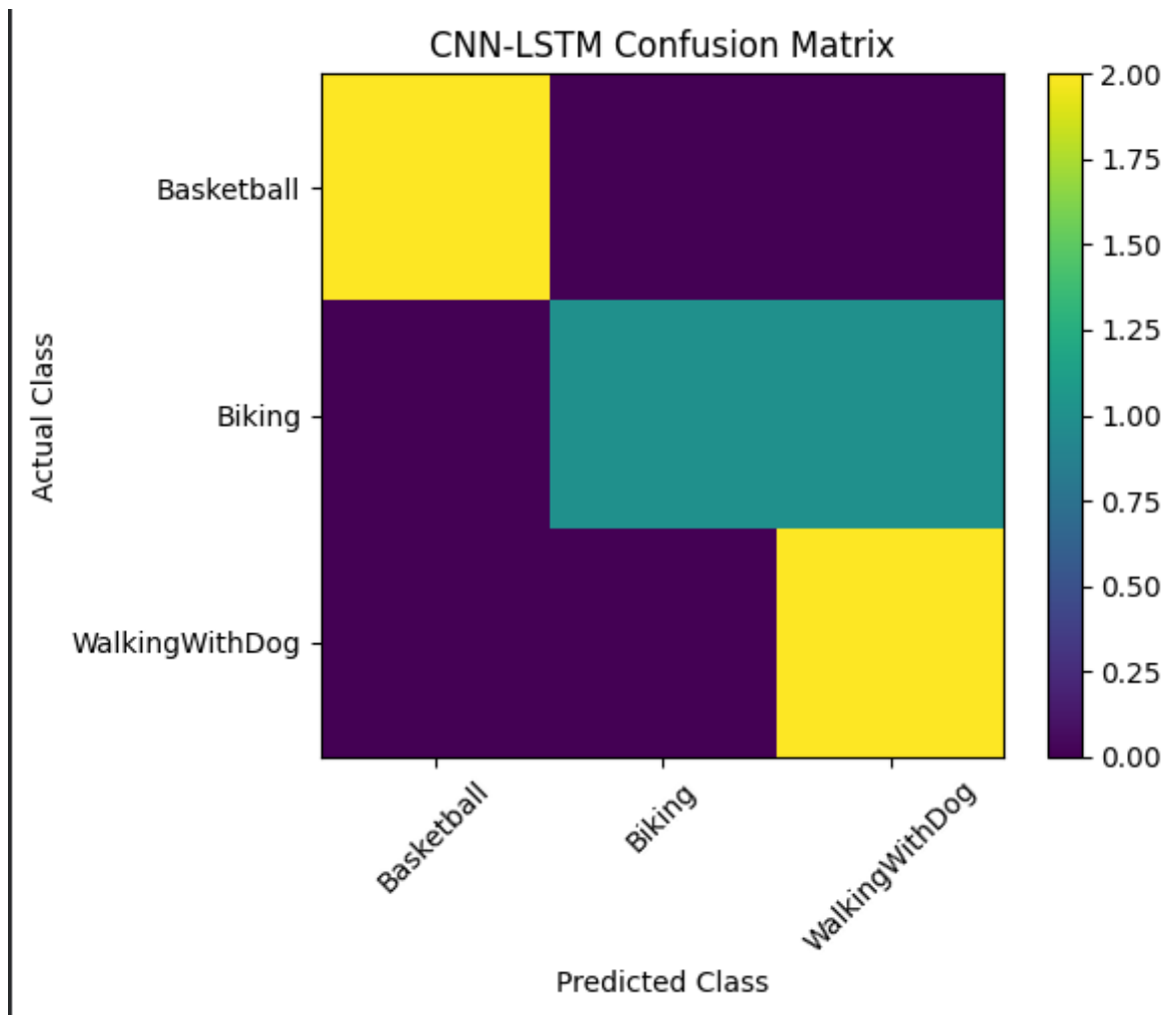


Figure 15: Confusion matrix for video action recognition.

#### Inference:

diagonal values show the correctly predicted video actions. The values outside the diagonal show the actions which are wrongly classified. We can see that biking is confused with walking with dog as the block is blue.

## 22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.  
For a simple laboratory task, construct a synthetic reversal dataset.  
Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

## 23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

**Expected output:**

[5, 1, 8, 3].

Students must display at least five test examples in the report.

| Sample | Input Sequence | Predicted Output |
|--------|----------------|------------------|
| 1      | [1, 4, 6, 2]   | [2, 6, 4, 1]     |
| 2      | [6, 8, 4, 9]   | [9, 4, 8, 6]     |
| 3      | [2, 9, 2, 9]   | [9, 2, 9, 2]     |
| 4      | [1, 7, 7, 9]   | [9, 7, 7, 1]     |
| 5      | [9, 3, 6, 4]   | [4, 6, 3, 9]     |

## 24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

**Required inference:**

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

**Inference:**

sequence level accuracy is lower because a sequence is correct only when all tokens are predicted correctly. so, even one incorrect token makes the entire sequence incorrect.

## 25. Consolidated Results

Students must complete the following table.

| Model        | Accuracy | Precision | Recall | F1     | Parameters |
|--------------|----------|-----------|--------|--------|------------|
| RNN          | 71.39%   | 71.13%    | 70.28% | 70.54% | 1,974      |
| LSTM         | 89.58%   | 89.54%    | 89.79% | 89.62% | 6,006      |
| GRU          | 89.72%   | 89.79%    | 89.84% | 89.74% | 4,758      |
| CNN-LSTM/GRU | 83.33%   | 88.89%    | 83.33% | 82.22% | 13,194     |

A separate table must be provided for the sequence-to-sequence task. **Inference:**

RNN gives 71.39

## 26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

## 27. Discussion Questions with Answers

1. **What is a recurrent neural network?**

A recurrent neural network is specially designed to work for sequential data as convolution neural network fails at it. so, it has a hidden state which stores all the past information. This hidden state is added to the current state without losing information about previous time steps to produce the current output.

2. **What information is represented by the hidden state?**

Hidden state acts like a memory for recurrent neural network. It stores the information about previous time steps that is necessary for the sequence.

3. **Why is sequence order important?**

sequence order is important because the meaning of data depends on the order. It helps RNN in capturing temporal or sequential relationships.

4. **Explain the difference between a feed-forward network and an RNN.**

A feed forward network takes inputs independently without the information about the relationship between them. RNN has an hidden state which stores the information about previous steps which helps in processing current inputs and it's relation between previous information.

5. **Explain Backpropagation Through Time.**

Backpropagation through time is a training algorithm used for RNN. We backpropagate through time steps and error is calculated and weights are updated at every time step.

6. **Why can gradients vanish during BPTT?**

Gradients are vanished as they are multiplied by smaller values multiple times. For long sequences we multiply more times which makes the gradient very small. This results in less learning at the starting layers.

7. **Why can gradients explode during BPTT?**

Gradients are vanished as they are multiplied by larger values multiple times. For long sequences we multiply more times which makes the gradient very high. This results in unstable learning at the starting layers.

8. **What are long-term dependencies?**

long term dependencies occur when we need information from previous time steps to predict something that is at very far time steps in a sequence.

9. **What is the role of the LSTM cell state?**

LSTM cell state acts like a long term memory for the model. It allows long term information to flow through the upcoming states of the model. The gates present help in controlling what and how much information to flow.

10. **What are the forget, input and output gates in LSTM?**

forget gate decides what information from the previous cell state should be removed. Input gate controls what new information should be stored. The output gate controls what information from the cell state should be given to the hidden state.

11. **What are the reset and update gates in GRU?**

The reset gate controls how much previous hidden state information should be ignored when generating new information. update gate controls how much previous hidden state information should be restored and how much new information should be taken into consideration.

12. **Compare LSTM and GRU.**

LSTM has one cell state and three gates(forget,input,output). GRU has only one hidden state and two gates(reset,update). so we can see that GRU is simpler and has less parameters but LSTM has more control on long term memory.

13. **Why does GRU generally use a simpler gating mechanism than LSTM?**

GRU combines some of the functions of LSTM's gates and does not have a separate cell state. This says that it is simpler with less number of parameters.

14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**

They should be compared using the same experimental settings because we can compare differences in output ,efficiency and accuracy that is caused by model type rather than difference in experimental setting.

15. **What does the confusion matrix reveal that accuracy alone does not?**

A confusion matrix tell us about predictions on every specific class. It gives information about which classes are being confused with each other by the model.

16. **Why should macro F1-score be reported for multi-class classification?**

macro F1-score calculates the F1-score for each class independently and then takes their average. It gives equal importance to every class so it is helpful when classes are imbalanced.

17. **What is the significance of sequence length?**

sequence length determines how many consecutive time steps are provided to the model.If the sequence length increases then the sequential or temporal information is passed to the next step but it increases the computational cost making learning difficult.

18. **Why can a CNN be used as a feature extractor for video?**

CNN can extract spatial features from individual video frames.We can use RNN to capture the information about transition of the image frames as it is temporal data.

19. **What type of information is extracted by the CNN?**

CNN extracts spatial information such as edges,shapes,textures,objects from the image.

20. **What type of information is learned by the recurrent network?**

The recurrent network learns temporal and sequential information.This includes transitions, order of events in time and relationships.

21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**

each video frame gives a feature vector as output from the CNN.These are arranged according to their order to form a sequence so that LSTM or GRU can then learn the temporal relationships between the frames.

22. **Explain the encoder and decoder in sequence-to-sequence learning.**

The encoder processes the input sequence and converts it into a representation containing information about the sequence.The decoder uses this representation to generate the output sequence step by step.

23. **What is teacher forcing?**

In the previous models,the predicted output is given as input to the other steps(layers) but in teacher forcing we give decoder the actual previous output rather than predicted outputs during training.

24. **Differentiate token accuracy and sequence accuracy.**

Token accuracy measures the percentage of how many individual tokens are predicted correctly.Sequence accuracy measures the percentage of complete sequences for which every token is predicted correctly.

25. **Why must the test set remain untouched during model selection?**

The test set should remain untouched because it is used to predict the unknown data so that we can accurately know how the model performs on actual deployment.This is unbiased way of testing.

## 28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
5. Change the sequence length and study the effect on computational cost.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.
7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

## 29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

### 30. Required Image Files

Place these image files in the same folder as the L<sup>A</sup>T<sub>E</sub>X source file. If a file is missing, the document will display a placeholder box.

| Image File                            | Purpose                     |
|---------------------------------------|-----------------------------|
| sensor_signal_vs_time.png             | Sensor signal visualization |
| rnn_training_validation_loss.png      | RNN loss curves             |
| lstm_training_validation_loss.png     | LSTM loss curves            |
| gru_training_validation_loss.png      | GRU loss curves             |
| rnn_training_validation_accuracy.png  | RNN accuracy curves         |
| lstm_training_validation_accuracy.png | LSTM accuracy curves        |
| gru_training_validation_accuracy.png  | GRU accuracy curves         |
| rnn_confusion_matrix.png              | RNN confusion matrix        |
| lstm_confusion_matrix.png             | LSTM confusion matrix       |
| gru_confusion_matrix.png              | GRU confusion matrix        |
| model_performance_comparison.png      | Model comparison            |
| sequence_length_vs_f1.png             | Sequence length vs F1       |
| video_sample_frames.png               | Sample video frames         |
| video_training_validation_curves.png  | Video training curves       |
| video_confusion_matrix.png            | Video confusion matrix      |

### 31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>

**GitHub Repository:** GitHub repository